the binding because named elements become fields within the page or user control that represents the root binding source.

Collections

If the data source is a collection, then a property path can specify items in the collection by their position or index. For example, "Teams[0].Players", where the literal "[]" encloses the "0" that requests the first item in a zero-indexed collection.

To use an indexer, the model needs to implement **IList<T>** or **IVector<T>** on the type of the property that is going to be indexed. (Note that IReadOnlyList<T> and IVectorView<T> do not support the indexer syntax.) If the type of the indexed property supports **INotifyCollectionChanged** or **IObservableVector** and the binding is OneWay or TwoWay, then it will register and listen for change notifications on those interfaces. The change detection logic will update based on all collection changes, even if that doesn't affect the specific indexed value. This is because the listening logic is common across all instances of the collection.

If the data source is a Dictionary or Map, then a property path can specify items in the collection by their string name. For example **<TextBlock Text="{x:Bind Players['John Smith']}" />** will look for an item in the dictionary named "John Smith". The name needs to be enclosed in quotes, and either single or double quotes can be used. Hat (^) can be used to escape quotes in strings. It's usually easiest to use alternate quotes from those used for the XAML attribute. (Note that IReadOnlyDictionary<T> and IMapView<T> do not support the indexer syntax.)

To use a string indexer, the model needs to implement **IDictionary<string, T>** or **IMap<string, T>** on the type of the property that is going to be indexed. If the type of the indexed property supports **IObservableMap** and the binding is OneWay or TwoWay, then it will register and listen for change notifications on those interfaces. The change detection logic will update based on all collection changes, even if that doesn't affect the specific indexed value. This is because the listening logic is common across all instances of the collection.

Attached Properties

To bind to attached properties, you need to put the class and property name into parentheses after the dot. For example **Text="{x:Bind Button22.(Grid.Row)}"**. If the property is not declared in a Xaml namespace, then you will need to prefix it with an xml namespace, which you should map to a code namespace at the head of the document.

Casting

Compiled bindings are strongly typed, and will resolve the type of each step in a path. If the type returned doesn't have the member, it will fail at compile time. You can specify a cast to tell binding the real type of the object.

In the following case, **obj** is a property of type object, but contains a text box, so we can use either **Text="{x:Bind ((TextBox)obj).Text}"** or **Text="{x:Bind obj.(TextBox.Text)}"**.

The **groups3** field in **Text="{x:Bind ((data:SampleDataGroup)groups3[0]).Title}"** is a dictionary of objects, so you must cast it to **data:SampleDataGroup**. Note the use of the xml **data:** namespace prefix for mapping the object type to a code namespace that isn't part of the default XAML namespace.

Note: The C#-style cast syntax is more flexible than the attached property syntax, and is the recommended syntax going forward.

Pathless casting

The native bind parser doesn't provide a keyword to represent `this` as a function parameter, but it does support pathless casting (for example, `{x:Bind (x:String)}`), which can be used as a function parameter. Therefore, `{x:Bind MethodName((namespace:TypeOfThis))}` is a valid way to perform what is conceptually equivalent to `{x:Bind MethodName(this)}`.

Example:

`Text="{x:Bind local:MainPage.GenerateSongTitle((local:SongItem))}"`

XAML

```
<Page
    x:Class="AppSample.MainPage"
    ...
    xmlns:local="using:AppSample">

    <Grid>
        <ListView ItemsSource="{x:Bind Songs}">
            <ListView.ItemTemplate>
                <DataTemplate x:DataType="local:SongItem">
                    <TextBlock
                        Margin="12"
                        FontSize="40"
                        Text="{x:Bind
local:MainPage.GenerateSongTitle((local:SongItem))}" />
                </DataTemplate>
            </ListView.ItemTemplate>
```

```
        </ListView>
    </Grid>
</Page>
```

C#

```
namespace AppSample
{
    public class SongItem
    {
        public string TrackName { get; private set; }
        public string ArtistName { get; private set; }

        public SongItem(string trackName, string artistName)
        {
            ArtistName = artistName;
            TrackName = trackName;
        }
    }

    public sealed partial class MainPage : Page
    {
        public List<SongItem> Songs { get; }
        public MainPage()
        {
            Songs = new List<SongItem>()
            {
                new SongItem("Track 1", "Artist 1"),
                new SongItem("Track 2", "Artist 2"),
                new SongItem("Track 3", "Artist 3")
            };

            this.InitializeComponent();
        }

        public static string GenerateSongTitle(SongItem song)
        {
            return $"{song.TrackName} - {song.ArtistName}";
        }
    }
}
```

Functions in binding paths

Starting in Windows 10, version 1607, **{x:Bind}** supports using a function as the leaf step of the binding path. This is a powerful feature for databinding that enables several scenarios in markup. See function bindings for details.

Event Binding

Event binding is a unique feature for compiled binding. It enables you to specify the handler for an event using a binding, rather than it having to be a method on the code behind. For example: **Click="{x:Bind rootFrame.GoForward}"**.

For events, the target method must not be overloaded and must also:

- Match the signature of the event.
- OR have no parameters.
- OR have the same number of parameters of types that are assignable from the types of the event parameters.

In generated code-behind, compiled binding handles the event and routes it to the method on the model, evaluating the path of the binding expression when the event occurs. This means that, unlike property bindings, it doesn't track changes to the model.

For more info about the string syntax for a property path, see Property-path syntax, keeping in mind the differences described here for **{x:Bind}**.

Properties that you can set with {x:Bind}

{x:Bind} is illustrated with the *bindingProperties* placeholder syntax because there are multiple read/write properties that can be set in the markup extension. The properties can be set in any order with comma-separated *propName=value* pairs. Note that you cannot include line breaks in the binding expression. Some of the properties require types that don't have a type conversion, so these require markup extensions of their own nested within the **{x:Bind}**.

These properties work in much the same way as the properties of the **Binding** class.

Expand table

Property	Description
Path	See the Property path section above.
Converter	Specifies the converter object that is called by the binding engine. The converter can be set in XAML, but only if you refer to an object instance that you've assigned in a {StaticResource} markup extension reference to that object in the resource dictionary.
ConverterLanguage	Specifies the culture to be used by the converter. (If you're setting **ConverterLanguage** you should also be setting **Converter**.) The culture is

Property	Description
	set as a standards-based identifier. For more info, see ConverterLanguage.
ConverterParameter	Specifies the converter parameter that can be used in converter logic. (If you're setting **ConverterParameter** you should also be setting **Converter**.) Most converters use simple logic that get all the info they need from the passed value to convert, and don't need a **ConverterParameter** value. The **ConverterParameter** parameter is for moderately advanced converter implementations that have more than one logic that keys off what's passed in **ConverterParameter**. You can write a converter that uses values other than strings but this is uncommon, see Remarks in ConverterParameter for more info.
FallbackValue	Specifies a value to display when the source or path cannot be resolved.
Mode	Specifies the binding mode, as one of these strings: "OneTime", "OneWay", or "TwoWay". The default is "OneTime". Note that this differs from the default for **{Binding}**, which is "OneWay" in most cases.
TargetNullValue	Specifies a value to display when the source value resolves but is explicitly **null**.
BindBack	Specifies a function to use for the reverse direction of a two-way binding.
UpdateSourceTrigger	Specifies when to push changes back from the control to the model in TwoWay bindings. The default for all properties except TextBox.Text is PropertyChanged; TextBox.Text is LostFocus.

> ⓘ Note
>
> If you're converting markup from **{Binding}** to **{x:Bind}**, then be aware of the differences in default values for the **Mode** property. x:DefaultBindMode can be used to change the default mode for x:Bind for a specific segment of the markup tree. The mode selected will apply any x:Bind expressions on that element and its children, that do not explicitly specify a mode as part of the binding. OneTime is more performant than OneWay as using OneWay will cause more code to be generated to hookup and handle the change detection.

Remarks

Because **{x:Bind}** uses generated code to achieve its benefits, it requires type information at compile time. This means that you cannot bind to properties where you do not know the type ahead of time. Because of this, you cannot use **{x:Bind}** with the **DataContext** property, which is of type **Object**, and is also subject to change at run time.

When using **{x:Bind}** with data templates, you must indicate the type being bound to by setting an **x:DataType** value, as shown in the Examples section. You can also set the type to an interface or base class type, and then use casts if necessary to formulate a full expression.

Compiled bindings depend on code generation. So if you use **{x:Bind}** in a resource dictionary then the resource dictionary needs to have a code-behind class. See Resource dictionaries with {x:Bind} for a code example.

Pages and user controls that include Compiled bindings will have a "Bindings" property in the generated code. This includes the following methods:

- **Update()** - This will update the values of all compiled bindings. Any one-way/Two-Way bindings will have the listeners hooked up to detect changes.
- **Initialize()** - If the bindings have not already been initialized, then it will call Update() to initialize the bindings
- **StopTracking()** - This will unhook all listeners created for one-way and two-way bindings. They can be re-initialized using the Update() method.

> **Note**
>
> Starting in Windows 10, version 1607, the XAML framework provides a built in Boolean to Visibility converter. The converter maps **true** to the **Visible** enumeration value and **false** to **Collapsed** so you can bind a Visibility property to a Boolean without creating a converter. Note that this is not a feature of function binding, only property binding. To use the built in converter, your app's minimum target SDK version must be 14393 or later. You can't use it when your app targets earlier versions of Windows 10. For more info about target versions, see Version adaptive code.

Tip If you need to specify a single curly brace for a value, such as in **Path** or **ConverterParameter**, precede it with a backslash: `\{`. Alternatively, enclose the entire string that contains the braces that need escaping in a secondary quotation set, for example `ConverterParameter='{Mix}'`.

Converter, **ConverterLanguage** and **ConverterLanguage** are all related to the scenario of converting a value or type from the binding source into a type or value that is compatible with the binding target property. For more info and examples, see the "Data conversions" section of Data binding in depth.

{x:Bind} is a markup extension only, with no way to create or manipulate such bindings programmatically. For more info about markup extensions, see XAML overview.

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

{Binding} markup extension

Article • 10/20/2022

> **Note**
>
> A new binding mechanism is available for Windows 10, which is optimized for performance and developer productivity. See **{x:Bind} markup extension**.

> **Note**
>
> For general info about using data binding in your app with **{Binding}** (and for an all-up comparison between **{x:Bind}** and **{Binding}**), see **Data binding in depth**.

The **{Binding}** markup extension is used to data bind properties on controls to values coming from a data source such as code. The **{Binding}** markup extension is converted at XAML load time into an instance of the **Binding** class. This binding object gets a value from a property on a data source, and pushes it to the property on the control. The binding object can optionally be configured to observe changes in the value of the data source property and update itself based on those changes. It can also optionally be configured to push changes to the control value back to the source property. The property that is the target of a data binding must be a dependency property. For more info, see Dependency properties overview.

{Binding} has the same dependency property precedence as a local value, and setting a local value in imperative code removes the effect of any **{Binding}** set in markup.

XAML attribute usage

syntax

```
<object property="{Binding}" .../>
-or-
<object property="{Binding propertyPath}" .../>
-or-
<object property="{Binding bindingProperties}" .../>
-or-
<object property="{Binding propertyPath, bindingProperties}" .../>
```

Term	Description

Term	Description
propertyPath	A string that specifies the property path for the binding. More info is in the Property path section below.
bindingProperties	*propName=value*[, *propName=value*]* One or more binding properties that are specified using a name/value pair syntax.
propName	The string name of the property to set on the **Binding** object. For example, "Converter".
value	The value to set the property to. The syntax of the argument depends on the property of Properties of the Binding class that can be set with {Binding} section below.

Property path

Path describes the property that you're binding to (the source property). Path is a positional parameter, which means you can use the parameter name explicitly (`{Binding Path=EmployeeID}`), or you can specify it as the first unnamed parameter (`{Binding EmployeeID}`).

The type of **Path** is a property path, which is a string that evaluates to a property or sub-property of either your custom type or a framework type. The type can be, but does not need to be, a **DependencyObject**. Steps in a property path are delimited by dots (.), and you can include multiple delimiters to traverse successive sub-properties. Use the dot delimiter regardless of the programming language used to implement the object being bound to.

For example, to bind UI to an employee object's first name property, your property path might be "Employee.FirstName". If you are binding an items control to a property that contains an employee's dependents, your property path might be "Employee.Dependents", and the item template of the items control would take care of displaying the items in "Dependents".

If the data source is a collection, then a property path can specify items in the collection by their position or index. For example, "Teams[0].Players", where the literal "[]" encloses the "0" that specifies the first item in a collection.

When using an **ElementName** binding to an existing **DependencyObject**, you can use attached properties as part of the property path. To disambiguate an attached property so that the intermediate dot in the attached property name is not considered a step into

a property path, put parentheses around the owner-qualified attached property name; for example, `(AutomationProperties.Name)`.

A property path intermediate object is stored as a **PropertyPath** object in a run-time representation, but most scenarios won't need to interact with a **PropertyPath** object in code. You can usually specify the binding info you need using XAML.

For more info about the string syntax for a property path, property paths in animation feature areas, and constructing a **PropertyPath** object, see Property-path syntax.

Properties of the Binding class that can be set with {Binding}

{Binding} is illustrated with the *bindingProperties* placeholder syntax because there are multiple read/write properties of a **Binding** that can be set in the markup extension. The properties can be set in any order with comma-separated *propName=value* pairs. Some of the properties require types that don't have a type conversion, so these require markup extensions of their own nested within the **{Binding}**.

Property	Description
Path	See the Property path section above.
Converter	Specifies a converter object that is called by the binding engine. The converter can be set in markup using the {StaticResource} markup extension to reference to that object from a resource dictionary.
ConverterLanguage	Specifies the culture to be used by the converter. (If you're setting **Converter**.) The culture is set as a standards-based identifier. For more info, see **ConverterLanguage**
ConverterParameter	Specifies a converter parameter that can be used in converter logic. (If you're setting **Converter**.) Most converters use simple logic that get all the info they need from the passed value to convert, and don't need a **ConverterParameter** value. The **ConverterParameter** parameter is for more complex converter implementations that have conditional logic that keys off what's passed in **ConverterParameter**. You can write a converter that uses values other than strings but this is uncommon, see Remarks in **ConverterParameter** for more info.
ElementName	Specifies a data source by referencing another element in the same XAML construct that has a **Name** property or x:Name attribute. This is often use to share related values or use sub-properties of one UI element to provide a specific value for another element, for example in a XAML control template.

Property	Description
FallbackValue	Specifies a value to display when the source or path cannot be resolved.
Mode	Specifies the binding mode, as one of these values: "OneTime", "OneWay", or "TwoWay". These correspond to the constant names of the BindingMode enumeration. The default is "OneWay". Note that this differs from the default for **{x:Bind}**, which is "OneTime".
RelativeSource	Specifies a data source by describing the position of the binding source relative to the position of the binding target. This is most often used in bindings within XAML control templates. Setting the {RelativeSource} markup extension.
Source	Specifies the object data source. Within the **Binding** markup extension, the **Source** property requires an object reference, such as a {StaticResource} markup extension reference. If this property is not specified, the acting data context specifies the source. It's more typical to not specify a Source value in individual bindings, and instead to rely on the shared **DataContext** for multiple bindings. For more info see **DataContext** or Data binding in depth.
TargetNullValue	Specifies a value to display when the source value resolves but is explicitly **null**.
UpdateSourceTrigger	Specifies the timing of binding source updates. If unspecified, the default is **Default**.

Note If you're converting markup from **{x:Bind}** to **{Binding}**, then be aware of the differences in default values for the **Mode** property.

Converter, ConverterLanguage and **ConverterLanguage** are all related to the scenario of converting a value or type from the binding source into a type or value that is compatible with the binding target property. For more info and examples, see the "Data conversions" section of Data binding in depth.

> **Note**
>
> Starting in Windows 10, version 1607, the XAML framework provides a built in Boolean to Visibility converter. The converter maps **true** to the **Visible** enumeration value and **false** to **Collapsed** so you can bind a Visibility property to a Boolean without creating a converter. To use the built in converter, your app's minimum target SDK version must be 14393 or later. You can't use it when your app targets earlier versions of Windows 10. For more info about target versions, see **Version adaptive code**.

Source, RelativeSource, and ElementName specify a binding source, so they are mutually exclusive.

Tip If you need to specify a single curly brace for a value, such as in Path or ConverterParameter, then precede it with a backslash: `\{`. Alternatively, enclose the entire string that contains the braces that need escaping in a secondary quotation set, for example `ConverterParameter='{Mix}'`.

Examples

XML

```
<!-- binding a UI element to a view model -->
<Page ... >
    <Page.DataContext>
        <local:BookstoreViewModel/>
    </Page.DataContext>

    <GridView ItemsSource="{Binding BookSkus}" SelectedItem="{Binding
SelectedBookSku, Mode=TwoWay}" ... />
</Page>
```

XML

```
<!-- binding a UI element to another UI element -->
<Page ... >
    <Page.Resources>
        <local:S2Formatter x:Key="GradeConverter"/>
    </Page.Resources>

    <Slider x:Name="sliderValueConverter" ... />
    <TextBox Text="{Binding Path=Value, ElementName=sliderValueConverter,
        Mode=OneWay,
        Converter={StaticResource GradeConverter}}"/>
</Page>
```

The second example sets four different Binding properties: ElementName, Path, Mode and Converter. **Path** in this case is shown explicitly named as a **Binding** property. The **Path** is evaluated to a data binding source that is another object in the same run-time object tree, a Slider named `sliderValueConverter`.

Note how the Converter property value uses another markup extension, {StaticResource} markup extension, so there are two nested markup extension usages here. The inner one is evaluated first, so that once the resource is obtained there's a

practical **IValueConverter** (a custom class that's instantiated by the `local:S2Formatter` element in resources) that the binding can use.

Tools support

Microsoft IntelliSense in Microsoft Visual Studio displays the properties of the data context while authoring **{Binding}** in the XAML markup editor. As soon as you type "{Binding", data context properties appropriate for **Path** are displayed in the dropdown. IntelliSense also helps with the other properties of **Binding**. For this to work, you must have either the data context or the design-time data context set in the markup page. **Go To Definition** (F12) also works with **{Binding}**. Alternatively, you can use the data binding dialog.

{x:Null} markup extension

Article • 10/20/2022

In XAML markup, specifies a **null** value for a property.

XAML attribute usage

syntax

```
<object property="{x:Null}" .../>
```

Remarks

null is the null reference keyword for C# and C++. The Microsoft Visual Basic keyword for a null reference is **Nothing**.

The initial default value can vary between dependency properties, and it is not necessarily **null**. Further, many dependency properties will not accept **null** as a value (whether through markup or code) due to their internal implementation. In such cases, setting a XAML attribute value with **{x:Null}** can result in a parser exception.

Some Windows Runtime types are nullable. In cases where a nullable type does not already have **null** as the default, you could use **{x:Null}** in XAML to set to the **null** value. If using Visual C++ component extensions (C++/CX), nullable types are represented as Platform::IBox<T>. If using Microsoft .NET languages, nullable types are represented as Nullable<T>.

Related topics

- Nullable<T>
- IReference<T>

XAML intrinsic data types

Article • 10/20/2022

XAML for the Windows Runtime provides language-level support for several data types that are frequently used primitives in the common language runtime (CLR) and in other programming languages such as C++.

The most common place you'll see XAML intrinsic data type usages is when resources are defined in a XAML resource dictionary. You might define constants there, for example numbers that you use for multiple values. Or you might use a storyboarded animation that animates using a string or Boolean value, and you'll then need a XAML object element representing the string or Boolean to fill the keyframe of your ObjectAnimationUsingKeyFrames definition. The Windows Runtime default XAML templates use both these techniques.

XAML for the Windows Runtime provides language-level support for these types.

XAML primitive	Description
x:Boolean	For CLR support, corresponds to Boolean. XAML parses values for **x:Boolean** as case insensitive. Note that "x:Bool" is not an accepted alternative.
x:String	For CLR support, corresponds to String. Encoding for the string defaults to the surrounding XML encoding.
x:Double	For CLR support, corresponds to Double. In addition to the numeric values, text syntax for **x:Double** permits the token "NaN", which is how "Auto" for layout behavior can be stored as a resource value. The tokens are treated as case sensitive. You can use scientific notation, for example "1+E06" for `1,000,000`.
x:Int32	For CLR support, corresponds to Int32. **x:Int32** is treated as signed, and you can include the minus ("-") symbol for a negative integer. In XAML, the absence of a sign in text syntax implies a positive signed value.

These XAML language primitives are generally the only cases in which you define an object element that uses the **x:** prefix in your XAML. All other XAML language features are typically used in attribute form, or as a markup extension.

Note By convention, the language primitives for XAML and all other XAML language elements are shown with the "x:" prefix. This is how XAML language elements are typically used in real-world markup. This convention is followed in the documentation for XAML and also in the XAML specification.

Other XAML primitives

The XAML 2009 specification notes other XAML language-level primitives such as **x:Uri** and **x:Single**. Unless listed in the table in this topic, other XAML language primitives as defined by other XAML vocabularies or by the XAML 2009 specification are not currently supported in XAML for the Windows Runtime.

Note Dates and times (properties that use **DateTime** or **DateTimeOffset**, **TimeSpan** or **System.TimeSpan**) aren't settable with a XAML primitive. These properties generally aren't settable in XAML at all, because there's no default from-string conversion behavior in the Windows Runtime XAML parser for dates and times. For initialization values of any date and time properties, you'll have to use code-behind that runs when a page or element loads.

Related topics

- XAML overview
- XAML syntax guide
- Storyboarded animations

{CustomResource} markup extension

Article • 10/20/2022

Provides a value for any XAML attribute by evaluating a reference to a resource that comes from a custom resource-lookup implementation. Resource lookup is performed by a CustomXamlResourceLoader class implementation.

XAML attribute usage

syntax

```
<object property="{CustomResource key}" .../>
```

XAML values

Term	Description
key	The key for the requested resource. How the key is initially assigned is specific to the implementation of the CustomXamlResourceLoader class that is currently registered for use.

Remarks

CustomResource is a technique for obtaining values that are defined elsewhere in a custom resource repository. This technique is relatively advanced and isn't used by most Windows Runtime app scenarios.

How a **CustomResource** resolves to a resource dictionary is not described in this topic, because that can vary widely depending on how CustomXamlResourceLoader is implemented.

The GetResource method of the CustomXamlResourceLoader implementation is called by the Windows Runtime XAML parser whenever it encounters a `{CustomResource}` usage in markup. The *resourceId* that is passed to **GetResource** comes from the *key* argument, and the other input parameters come from context, such as which property the usage is applied to.

A `{CustomResource}` usage doesn't work by default (the base implementation of GetResource is incomplete). To make a valid `{CustomResource}` reference, you must

perform each of these steps:

1. Derive a custom class from **CustomXamlResourceLoader** and override **GetResource** method. Do not call base in the implementation.
2. Set **CustomXamlResourceLoader.Current** to reference your class in initialization logic. This must happen before any page-level XAML that includes the `{CustomResource}` extension usage is loaded. One place to set **CustomXamlResourceLoader.Current** is in the **Application** subclass constructor that's generated for you in the App.xaml code-behind templates.
3. Now you can use `{CustomResource}` extensions in the XAML that your app loads as pages, or from within XAML resource dictionaries.

CustomResource is a markup extension. Markup extensions are typically implemented when there is a requirement to escape attribute values to be other than literal values or handler names, and the requirement is more global than just putting type converters on certain types or properties. All markup extensions in XAML use the "{" and "}" characters in their attribute syntax, which is the convention by which a XAML processor recognizes that a markup extension must process the attribute.

Related topics

- ResourceDictionary and XAML resource references
- **CustomXamlResourceLoader**
- **GetResource**

{RelativeSource} markup extension

Article • 10/20/2022

Provides a means to specify the source of a binding in terms of a relative relationship in the run-time object graph.

XAML attribute usage (Self mode)

syntax

```
<Binding RelativeSource="{RelativeSource Self}" .../>
-or-
<object property="{Binding RelativeSource={RelativeSource Self} ...}" .../>
```

XAML attribute usage (TemplatedParent mode)

syntax

```
<Binding RelativeSource="{RelativeSource TemplatedParent}" .../>
-or-
<object property="{Binding RelativeSource={RelativeSource TemplatedParent}
...}" .../>
```

XAML values

Term	Description
{RelativeSource Self}	Produces a Mode value of **Self**. The target element should be used as the source for this binding. This is useful for binding one property of an element to another property on the same element.
{RelativeSource TemplatedParent}	Produces a ControlTemplate that is applied as the source for this binding. This is useful for applying runtime information to bindings at the template level.

Remarks

A Binding can set Binding.RelativeSource either as an attribute on a **Binding** object element or as a component within a {Binding} markup extension. This is why two different XAML syntaxes are shown.

RelativeSource is similar to {Binding} markup extension. It is a markup extension that is capable of returning instances of itself, and supporting a string-based construction that essentially passes an argument to the constructor. In this case, the argument being passed is the **Mode** value.

The **Self** mode is useful for binding one property of an element to another property on the same element, and is a variation on **ElementName** binding but does not require naming and then self-referencing the element. If you bind one property of an element to another property on the same element, either the properties must use the same property type, or you must also use a **Converter** on the binding to convert the values. For example, you could use **Height** as a source for **Width** without conversion, but you'd need a converter to use **IsEnabled** as a source for **Visibility**.

Here's an example. This **Rectangle** uses a {Binding} markup extension so that its **Height** and **Width** are always equal and it renders as a square. Only the Height is set as a fixed value. For this **Rectangle** its default **DataContext** is **null**, not **this**. So to establish the data context source to be the object itself (and enable binding to its other properties) we use the `RelativeSource={RelativeSource Self}` argument in the {Binding} markup extension usage.

XML

```
<Rectangle
  Fill="Orange" Width="200"
  Height="{Binding RelativeSource={RelativeSource Self}, Path=Width}"
/>
```

Another use of `RelativeSource={RelativeSource Self}` is as a way to set an object's **DataContext** to itself. For example, you may see this technique in some of the SDK examples where the **Page** class has been extended with a custom property that's already providing a ready-to-go view model for its own data binding such as: `<common:LayoutAwarePage ... DataContext="{Binding DefaultViewModel, RelativeSource={RelativeSource Self}}">`

Note The XAML usage for **RelativeSource** shows only the usage for which it is intended: setting a value for **Binding.RelativeSource** in XAML as part of a binding expression. Theoretically, other usages are possible if setting a property where the value is **RelativeSource**.

Related topics

- XAML overview

- Data binding in depth
- {Binding} markup extension
- **Binding**
- **RelativeSource**

{StaticResource} markup extension

Article • 10/20/2022

Provides a value for any XAML attribute by evaluating a reference to an already defined resource. Resources are defined in a ResourceDictionary, and a **StaticResource** usage references the key of that resource in the **ResourceDictionary**.

XAML attribute usage

syntax

```
<object property="{StaticResource key}" .../>
```

XAML values

Term	Description
key	The key for the requested resource. This key is initially assigned by the ResourceDictionary. A resource key can be any string defined in the XamlName Grammar.

Remarks

StaticResource is a technique for obtaining values for a XAML attribute that are defined elsewhere in a XAML resource dictionary. Values might be placed in a resource dictionary because they are intended to be shared by multiple property values, or because a XAML resource dictionary is used as a XAML packaging or factoring technique. An example of a XAML packaging technique is the theme dictionary for a control. Another example is merged resource dictionaries used for resource fallback.

StaticResource takes one argument, which specifies the key for the requested resource. A resource key is always a string in Windows Runtime XAML. For more info on how the resource key is initially specified, see x:Key attribute.

The rules by which a **StaticResource** resolves to an item in a resource dictionary are not described in this topic. That depends on whether the reference and the resource both exist in a template, whether merged resource dictionaries are used, and so on. For more info on how to define resources and properly use a **ResourceDictionary**, including sample code, see ResourceDictionary and XAML resource references.

Important A **StaticResource** must not attempt to make a forward reference to a resource that is defined lexically further within the XAML file. Attempting to do so is not supported. Even if the forward reference doesn't fail, trying to make one carries a performance penalty. For best results, adjust the composition of your resource dictionaries so that forward references are avoided.

Attempting to specify a **StaticResource** to a key that cannot resolve throws a XAML parse exception at run time. Design tools may also offer warnings or errors.

In the Windows Runtime XAML processor implementation, there is no backing class representation for **StaticResource** functionality. **StaticResource** is exclusively for use in XAML. The closest equivalent in code is to use the collection API of a **ResourceDictionary**, for example calling **Contains** or **TryGetValue**.

{ThemeResource} markup extension is a similar markup extension that references named resources in another location. The difference is that {ThemeResource} markup extension has the ability to return different resources depending on the system theme that's active. For more info see {ThemeResource} markup extension.

StaticResource is a markup extension. Markup extensions are typically implemented when there is a requirement to escape attribute values to be other than literal values or handler names, and the requirement is more global than just putting type converters on certain types or properties. All markup extensions in XAML use the "{" and "}" characters in their attribute syntax, which is the convention by which a XAML processor recognizes that a markup extension must process the attribute.

An example {StaticResource} usage

This example XAML is taken from the XAML data binding sample.

XML

```
<StackPanel Margin="5">
    <!-- Add converter as a resource to reference it from a Binding. -->
    <StackPanel.Resources>
        <local:S2Formatter x:Key="GradeConverter"/>
    </StackPanel.Resources>
    <TextBlock Style="{StaticResource BasicTextStyle}" Text="Percent grade:"
Margin="5" />
    <Slider x:Name="sliderValueConverter" Minimum="1" Maximum="100"
Value="70" Margin="5"/>
    <TextBlock Style="{StaticResource BasicTextStyle}" Text="Letter grade:"
Margin="5"/>
    <TextBox x:Name="tbValueConverterDataBound"
      Text="{Binding ElementName=sliderValueConverter, Path=Value,
Mode=OneWay,
```

```
        Converter={StaticResource GradeConverter}}" Margin="5" Width="150"/>
</StackPanel>
```

This particular example creates an object that's backed by a custom class, and creates it as a resource in a **ResourceDictionary**. To be a valid resource, this `local:S2Formatter` element must also have an **x:Key** attribute value. The value of the attribute is set to "GradeConverter".

The resource is then requested just a bit further into the XAML, where you see `{StaticResource GradeConverter}`.

Note how the {StaticResource} markup extension usage is setting a property of another markup extension {Binding} markup extension, so there's two nested markup extension usages here. The inner one is evaluated first, so that the resource is obtained first and can be used as a value. This same example is also shown in {Binding} markup extension.

Design-time tools support for the {StaticResource} markup extension

Microsoft Visual Studio 2013 can include possible key values in the Microsoft IntelliSense dropdowns when you use the **{StaticResource}** markup extension in a XAML page. For example, as soon as you type "{StaticResource", any of the resource keys from the current lookup scope are displayed in the IntelliSense dropdowns. In addition to the typical resources you'd have at page level (**FrameworkElement.Resources**) and app level (**Application.Resources**), you also see XAML theme resources, and resources from any extensions your project is using.

Once a resource key exists as part of any **{StaticResource}** usage, the **Go To Definition** (F12) feature can resolve that resource and show you the dictionary where it's defined. For the theme resources, this goes to generic.xaml for design time.

Related topics

- ResourceDictionary and XAML resource references
- **ResourceDictionary**
- x:Key attribute
- {ThemeResource} markup extension

{TemplateBinding} markup extension

Article • 10/20/2022

Links the value of a property in a control template to the value of some other exposed property on the templated control. **TemplateBinding** can only be used within a ControlTemplate definition in XAML.

XAML attribute usage

syntax

```
<object propertyName="{TemplateBinding sourceProperty}" .../>
```

XAML attribute usage (for Setter property in template or style)

syntax

```
<Setter Property="propertyName" Value="{TemplateBinding sourceProperty}"
.../>
```

XAML values

Term	Description
propertyName	The name of the property being set in the setter syntax. This must be a dependency property.
sourceProperty	The name of another dependency property that exists on the type being templated.

Remarks

Using **TemplateBinding** is a fundamental part of how you define a control template, either if you are a custom control author or if you are replacing a control template for existing controls. For more info, see Quickstart: Control templates.

It's fairly common for *propertyName* and *targetProperty* to use the same property name. In this case, a control might define a property on itself and forward the property to an existing and intuitively named property of one of its component parts. For example, a control that incorporates a **TextBlock** in its compositing, which is used to display the control's own **Text** property, might include this XAML as a part in the control template: `<TextBlock Text="{TemplateBinding Text}" .... />`

The types used as the value for the source property and the target property must match. There's no opportunity to introduce a converter when you're using **TemplateBinding**. Failing to match values results in an error when parsing the XAML. If you need a converter you can use the verbose syntax for a template binding such as: `{Binding RelativeSource={RelativeSource TemplatedParent}, Converter="..." ...}`

Attempting to use a **TemplateBinding** outside of a **ControlTemplate** definition in XAML will result in a parser error.

You can use **TemplateBinding** for cases where the templated parent value is also deferred as another binding. The evaluation for **TemplateBinding** can wait until any required runtime bindings have values.

A **TemplateBinding** is always a one-way binding. Both properties involved must be dependency properties.

TemplateBinding is a markup extension. Markup extensions are typically implemented when there is a requirement to escape attribute values to be other than literal values or handler names, and the requirement is more global than just putting type converters on certain types or properties. All markup extensions in XAML use the "{" and "}" characters in their attribute syntax, which is the convention by which a XAML processor recognizes that a markup extension must process the attribute.

Note In the Windows Runtime XAML processor implementation, there is no backing class representation for **TemplateBinding**. **TemplateBinding** is exclusively for use in XAML markup. There isn't a straightforward way to reproduce the behavior in code.

x:Bind in ControlTemplate

> ⓘ **Note**
>
> Using x:Bind in a ControlTemplate requires Windows 10, version 1809 (**SDK 17763**) or later. For more info about target versions, see **Version adaptive code**.

Starting with Windows 10, version 1809, you can use the **x:Bind** markup extension anywhere you use **TemplateBinding** in a **ControlTemplate**.

The TargetType property is required (not optional) on ControlTemplate when using **x:Bind**.

With **x:Bind** support, you can use both Function bindings as well as two-way bindings in a ControlTemplate.

In this example, the **TextBlock.Text** property evaluates to **Button.Content.ToString**. The TargetType on the ControlTemplate acts as the data source and accomplishes the same result as a TemplateBinding to parent.

XAML

```
<ControlTemplate TargetType="Button">
    <Grid>
        <TextBlock Text="{x:Bind Content, Mode=OneWay}"/>
    </Grid>
</ControlTemplate>
```

Related topics

- Quickstart: Control templates
- Data binding in depth
- **ControlTemplate**
- XAML overview
- Dependency properties overview

{ThemeResource} markup extension

Article • 10/20/2022

Provides a value for any XAML attribute by evaluating a reference to a resource, with additional system logic that retrieves different resources depending on the currently active theme. Similar to {StaticResource} markup extension, resources are defined in a ResourceDictionary, and a **ThemeResource** usage references the key of that resource in the **ResourceDictionary**.

XAML attribute usage

syntax

```
<object property="{ThemeResource key}" .../>
```

XAML values

Term	Description
key	The key for the requested resource. This key is initially assigned by the ResourceDictionary. A resource key can be any string defined in the XamlName Grammar.

Remarks

A **ThemeResource** is a technique for obtaining values for a XAML attribute that are defined elsewhere in a XAML resource dictionary. The markup extension serves the same basic purpose as the {StaticResource} markup extension. The difference in behavior versus {StaticResource} markup extension is that a **ThemeResource** reference can dynamically use different dictionaries as the primary lookup location, depending on which theme is currently being used by the system.

When the app first starts, any resource reference made by a **ThemeResource** reference is evaluated based on the theme in use at startup. But if the user subsequently changes the active theme at run-time, the system will re-evaluate every **ThemeResource** reference, retrieve a theme-specific resource that may be different, and redisplay the app with new resource values in all appropriate places in the visual tree. A **StaticResource** is determined at XAML load time / app startup and won't be re-evaluated at run-time. (There are other techniques such as visual states that reload

XAML dynamically, but those techniques operate at a higher level that the basic resource evaluation enabled by {StaticResource} markup extension).

ThemeResource takes one argument, which specifies the key for the requested resource. A resource key is always a string in Windows Runtime XAML. For more info on how the resource key is initially specified, see x:Key attribute.

For more info on how to define resources and properly use a **ResourceDictionary**, including sample code, see ResourceDictionary and XAML resource references.

Important As with **StaticResource**, a **ThemeResource** must not attempt to make a forward reference to a resource that is defined lexically further within the XAML file. Attempting to do so is not supported. Even if the forward reference doesn't fail, trying to make one carries a performance penalty. For best results, adjust the composition of your resource dictionaries so that forward references are avoided.

Attempting to specify a **ThemeResource** to a key that cannot resolve throws a XAML parse exception at run time. Design tools may also offer warnings or errors.

In the Windows Runtime XAML processor implementation, there is no backing class representation for **ThemeResource**. The closest equivalent in code is to use the collection API of a **ResourceDictionary**, for example calling **Contains** or **TryGetValue*.

ThemeResource is a markup extension. Markup extensions are typically implemented when there is a requirement to escape attribute values to be other than literal values or handler names, and the requirement is more global than just putting type converters on certain types or properties. All markup extensions in XAML use the "{" and "}" characters in their attribute syntax, which is the convention by which a XAML processor recognizes that a markup extension must process the attribute.

When and how to use {ThemeResource} rather than {StaticResource}

The rules by which a **ThemeResource** resolves to an item in a resource dictionary are generally the same as **StaticResource**. A **ThemeResource** lookup can extend into the ResourceDictionary files that are referenced in a ThemeDictionaries collection, but a **StaticResource** can do that also. The difference is that a **ThemeResource** can re-evaluate at run-time and a **StaticResource** can't.

The set of keys in each theme dictionary should provide the same set of keyed resources no matter which theme is active. If a given keyed resource exists in the **HighContrast** theme dictionary, then another resource with that name should also exist in **Light** and **Default**. If that isn't true, resource lookup might fail when the user switches themes and

your app won't look right. It is possible though that a theme dictionary can contain keyed resources that are only referenced from within the same scope to provide sub-values; these don't need to be equivalent in all themes.

In general you should place resources in theme dictionaries and make references to those resources using **ThemeResource** only when those values can change between themes or are supported by values that change. This is appropriate for these kinds of resources:

- Brushes, in particular colors for **SolidColorBrush**. These make up about 80% of the **ThemeResource** usages in the default XAML control templates (generic.xaml).
- Pixel values for borders, offsets, margin and padding and so on.
- Font properties such as **FontFamily** or **FontSize**.
- Complete templates for a limited number of controls that are usually system-styled and used for dynamic presentation, like **GridViewItem** and **ListViewItem**.
- Text display styles (usually to change font color, background, and possibly size).

The Windows Runtime provides a set of resources that are specifically intended to be referenced by **ThemeResource**. These are all listed as part of the XAML file themeresources.xaml, which is available in the include/winrt/xaml/design folder as part of the Windows Software Development Kit (SDK). For documentation on the theme brushes and additional styles that are defined in themeresources.xaml, see XAML theme resources. The brushes are documented in a table that tells you what color value each brush has for each of the three possible active themes.

The XAML definitions of visual states in a control template should use **ThemeResource** references whenever there's an underlying resource that might change because of a theme change. A system theme change won't typically also cause a visual state change. The resources need to use **ThemeResource** references in this case so that values can be re-evaluated for the still-active visual state. For example, if you have a visual state that changes a brush color of a particular UI part and one of its properties, and that brush color is different per-theme, you should use a **ThemeResource** reference for providing that property's value in the default template and also any visual state modification to that default template.

ThemeResource usages might be seen in a series of dependent values. For example, a **Color** value used by a **SolidColorBrush** that is also a keyed resource might use a **ThemeResource** reference. But any UI properties that use the keyed **SolidColorBrush** resource would also use a **ThemeResource** reference, so that it's specifically each **Brush**-type property that's enabling a dynamic value change when the theme changes.

Note `{ThemeResource}` and run-time resource evaluation on theme switching is supported in Windows 8.1 XAML but not supported in XAML for apps targeting

Windows 8.

System resources

Some theme resources reference system resource values as an underlying sub-value. A system resource is a special resource value that isn't found in any XAML resource dictionary. These values rely on behavior in Windows Runtime XAML support to forward values from the system itself, and represent them in a form that a XAML resource can reference. For example, there is a system resource named "SystemColorButtonFaceColor" that represents an RGB color. This color comes from the aspects of system colors and themes that aren't just specific to Windows Runtime and Windows Runtime apps.

System resources are often the underlying values for a high-contrast theme. The user is in control of the color choices for their high-contrast theme, and the user makes these choices using system features that also aren't specific to Windows Runtime apps. By referencing the system resources as **ThemeResource** references, the default behavior of the high-contrast themes for Windows Runtime apps can use these theme-specific values that are controlled by the user and exposed by the system. Also, the references are now marked for re-evaluation if the system detects a run-time theme change.

An example {ThemeResource} usage

Here's some example XAML taken from the default generic.xaml and themeresources.xaml files to illustrate how to use **ThemeResource**. We'll look at just one template (the default **Button**) and how two properties are declared (**Background** and **Foreground**) to be responsive to theme changes.

XML

```
    <!-- Default style for Windows.UI.Xaml.Controls.Button -->
    <Style TargetType="Button">
        <Setter Property="Background" Value="{ThemeResource
ButtonBackgroundThemeBrush}" />
        <Setter Property="Foreground" Value="{ThemeResource
ButtonForegroundThemeBrush}"/>
...
```

Here, the properties take a **Brush** value, and the reference to **SolidColorBrush** resources named `ButtonBackgroundThemeBrush` and `ButtonForegroundThemeBrush` are made using **ThemeResource**.

These same properties are also adjusted by some of the visual states for a **Button**. Notably, the background color changes when a button is clicked. Here too, the **Background** and **Foreground** animations in the visual state storyboard use **DiscreteObjectKeyFrame** objects and references to brushes with **ThemeResource** as the key frame value.

XML

```
<VisualState x:Name="Pressed">
  <Storyboard>
    <ObjectAnimationUsingKeyFrames Storyboard.TargetName="Border"
        Storyboard.TargetProperty="Background">
      <DiscreteObjectKeyFrame KeyTime="0" Value="{ThemeResource
ButtonPressedBackgroundThemeBrush}" />
    </ObjectAnimationUsingKeyFrames>
    <ObjectAnimationUsingKeyFrames Storyboard.TargetName="ContentPresenter"
         Storyboard.TargetProperty="Foreground">
       <DiscreteObjectKeyFrame KeyTime="0" Value="{ThemeResource
ButtonPressedForegroundThemeBrush}" />
    </ObjectAnimationUsingKeyFrames>
  </Storyboard>
</VisualState>
```

Each of these brushes is defined earlier in generic.xaml: they had to be defined prior to any templates using them to avoid XAML forward references. Here's those definitions, for the "Default" theme dictionary.

XML

```
    <ResourceDictionary.ThemeDictionaries>
        <ResourceDictionary x:Key="Default">
...
            <SolidColorBrush x:Key="ButtonBackgroundThemeBrush"
Color="Transparent" />
            <SolidColorBrush x:Key="ButtonForegroundThemeBrush"
Color="#FFFFFFFF" />
...
            <SolidColorBrush x:Key="ButtonPressedBackgroundThemeBrush"
Color="#FFFFFFFF" />
            <SolidColorBrush x:Key="ButtonPressedForegroundThemeBrush"
Color="#FF000000" />
...
```

Then each of the other theme dictionaries also has these brushes defined, for example:

XML

```
        <ResourceDictionary x:Key="HighContrast">
            <!-- High Contrast theme resources -->
```

```
...
            <SolidColorBrush x:Key="ButtonBackgroundThemeBrush" Color="
{ThemeResource SystemColorButtonFaceColor}" />
            <SolidColorBrush x:Key="ButtonForegroundThemeBrush" Color="
{ThemeResource SystemColorButtonTextColor}" />

...
            <SolidColorBrush x:Key="ButtonPressedBackgroundThemeBrush"
Color="{ThemeResource SystemColorButtonTextColor}" />
            <SolidColorBrush x:Key="ButtonPressedForegroundThemeBrush"
Color="{ThemeResource SystemColorButtonFaceColor}" />
```

Here the **Color** value is another **ThemeResource** reference to a system resource. If you reference a system resource, and you want it to change in response to a theme change, you should use **ThemeResource** to make the reference.

Windows 8 behavior

Windows 8 did not support the **ThemeResource** markup extension, it is available starting with Windows 8.1. Also, Windows 8 did not support dynamically switching the theme-related resources for a Windows Runtime app. The app had to be restarted in order to pick up the theme change for the XAML templates and styles. This isn't a good user experience, so apps are strongly encouraged to recompile and target Windows 8.1 so that they can use styles with **ThemeResource** usages and can dynamically switch themes when the user does. Apps that were compiled for Windows 8 but running on Windows 8.1 continue to use the Windows 8 behavior.

Design-time tools support for the {ThemeResource} markup extension

Microsoft Visual Studio 2013 can include possible key values in the Microsoft IntelliSense dropdowns when you use the **{ThemeResource}** markup extension in a XAML page. For example, as soon as you type "{ThemeResource", any of the resource keys from the XAML theme resources are displayed.

Once a resource key exists as part of any **{ThemeResource}** usage, the **Go To Definition** (F12) feature can resolve that resource and show you the generic.xaml for design time, where the theme resource is defined. Because theme resources are defined more than once (per-theme) **Go To Definition** takes you to the first definition found in the file, which is the definition for **Default**. If you want the other definitions you can search for the key name within the file and find the other themes' definitions.

Related topics

- ResourceDictionary and XAML resource references
- XAML theme resources
- **ResourceDictionary**
- x:Key attribute

Dependency properties overview

Article • 10/20/2022

This topic explains the dependency property system that is available when you write a Windows Runtime app using C++, C#, or Visual Basic along with XAML definitions for UI.

What is a dependency property?

A dependency property is a specialized type of property. Specifically it's a property where the property's value is tracked and influenced by a dedicated property system that is part of the Windows Runtime.

In order to support a dependency property, the object that defines the property must be a **DependencyObject** (in other words a class that has the **DependencyObject** base class somewhere in its inheritance). Many of the types you use for your UI definitions for a UWP app with XAML will be a **DependencyObject** subclass, and will support dependency properties. However, any type that comes from a Windows Runtime namespace that doesn't have "XAML" in its name won't support dependency properties; properties of such types are ordinary properties that won't have the property system's dependency behavior.

The purpose of dependency properties is to provide a systemic way to compute the value of a property based on other inputs (other properties, events and states that occur within your app while it runs). These other inputs might include:

- External input such as user preference
- Just-in-time property determination mechanisms such as data binding, animations and storyboards
- Multiple-use templating patterns such as resources and styles
- Values known through parent-child relationships with other elements in the object tree

A dependency property represents or supports a specific feature of the programming model for defining a Windows Runtime app with XAML for UI and C#, Microsoft Visual Basic or Visual C++ component extensions (C++/CX) for code. These features include:

- Data binding
- Styles
- Storyboarded animations

- "PropertyChanged" behavior; a dependency property can be implemented to provide callbacks that can propagate changes to other dependency properties
- Using a default value that comes from property metadata
- General property system utility such as ClearValue and metadata lookup

Dependency properties and Windows Runtime properties

Dependency properties extend basic Windows Runtime property functionality by providing a global, internal property store that backs all of the dependency properties in an app at run time. This is an alternative to the standard pattern of backing a property with a private field that's private in the property-definition class. You can think of this internal property store as being a set of property identifiers and values that exist for any particular object (so long as it's a DependencyObject). Rather than being identified by name, each property in the store is identified by a DependencyProperty instance. However, the property system mostly hides this implementation detail: you can usually access dependency properties by using a simple name (the programmatic property name in the code language you're using, or an attribute name when you're writing XAML).

The base type that provides the underpinnings of the dependency property system is DependencyObject. **DependencyObject** defines methods that can access the dependency property, and instances of a **DependencyObject** derived class internally support the property store concept we mentioned earlier.

Here is a summation of the terminology that we use in the documentation when discussing dependency properties:

Term	Description
Dependency property	A property that exists on a DependencyProperty identifier (see below). Usually this identifier is available as a static member of the defining **DependencyObject** derived class.
Dependency property identifier	A constant value to identify the property, it is typically public and read-only.
Property wrapper	The callable **get** and **set** implementations for a Windows Runtime property. Or, the language-specific projection of the original definition. A **get** property wrapper implementation calls GetValue, passing the relevant dependency property identifier.

The property wrapper is not just convenience for callers, it also exposes the dependency property to any process, tool or projection that uses Windows Runtime definitions for properties.

The following example defines a custom dependency property as defined for C#, and shows the relationship of the dependency property identifier to the property wrapper.

C#

```
public static readonly DependencyProperty LabelProperty =
DependencyProperty.Register(
  "Label",
  typeof(string),
  typeof(ImageWithLabelControl),
  new PropertyMetadata(null)
);


public string Label
{
    get { return (string)GetValue(LabelProperty); }
    set { SetValue(LabelProperty, value); }
}
```

> **Note**
>
> The preceding example is not intended as the complete example for how to create a custom dependency property. It is intended to show dependency property concepts for anyone that prefers learning concepts through code. For a more complete explanation of this example, see **Custom dependency properties**.

Dependency property value precedence

When you get the value of a dependency property, you are obtaining a value that was determined for that property through any one of the inputs that participate in the Windows Runtime property system. Dependency property value precedence exists so that the Windows Runtime property system can calculate values in a predictable way, and it's important that you be familiar with the basic precedence order too. Otherwise, you might find yourself in a situation where you're trying to set a property at one level of precedence but something else (the system, third-party callers, some of your own code) is setting it at another level, and you'll get frustrated trying to figure out which property value is used and where that value came from.

For example, styles and templates are intended to be a shared starting point for establishing property values and thus appearances of a control. But on a particular control instance you might want to change its value versus the common templated value, such as giving that control a different background color or a different text string as content. The Windows Runtime property system considers local values at higher precedence than values provided by styles and templates. That enables the scenario of having app-specific values overwrite the templates so that the controls are useful for your own use of them in app UI.

Dependency property precedence list

The following is the definitive order that the property system uses when assigning the run-time value for a dependency property. Highest precedence is listed first. You'll find more detailed explanations just past this list.

1. **Animated values:** Active animations, visual state animations, or animations with a HoldEnd behavior. To have any practical effect, an animation applied to a property must have precedence over the base (unanimated) value, even if that value was set locally.
2. **Local value:** A local value might be set through the convenience of the property wrapper, which also equates to setting as an attribute or property element in XAML, or by a call to the SetValue method using a property of a specific instance. If you set a local value by using a binding or a static resource, these each act in the precedence as if a local value was set, and bindings or resource references are erased if a new local value is set.
3. **Templated properties:** An element has these if it was created as part of a template (from a ControlTemplate or DataTemplate).
4. **Style setters:** Values from a Setter within styles from page or application resources.
5. **Default value:** A dependency property can have a default value as part of its metadata.

Templated properties

Templated properties as a precedence item do not apply to any property of an element that you declare directly in XAML page markup. The templated property concept exists only for objects that are created when the Windows Runtime applies a XAML template to a UI element and thus defines its visuals.

All the properties that are set from a control template have values of some kind. These values are almost like an extended set of default values for the control and are often associated with values you can reset later by setting the property values directly. Thus

the template-set values must be distinguishable from a true local value, so that any new local value can overwrite it.

> **Note**
>
> In some cases the template might override even local values, if the template failed to expose **{TemplateBinding} markup extension** references for properties that should have been settable on instances. This is usually done only if the property is really not intended to be set on instances, for example if it's only relevant to visuals and template behavior and not to the intended function or runtime logic of the control that uses the template.

Bindings and precedence

Binding operations have the appropriate precedence for whatever scope they're used for. For example, a {Binding} applied to a local value acts as local value, and a {TemplateBinding} markup extension for a property setter applies as a style setter does. Because bindings must wait until run-time to obtain values from data sources, the process of determining the property value precedence for any property extends into run-time as well.

Not only do bindings operate at the same precedence as a local value, they really are a local value, where the binding is the placeholder for a value that is deferred. If you have a binding in place for a property value, and you set a local value on it at run-time, that replaces the binding entirely. Similarly, if you call **SetBinding** to define a binding that only comes into existence at run-time, you replace any local value you might have applied in XAML or with previously executed code.

Storyboarded animations and base value

Storyboarded animations act on a concept of a *base value*. The base value is the value that's determined by the property system using its precedence, but omitting that last step of looking for animations. For example, a base value might come from a control's template, or it might come from setting a local value on an instance of a control. Either way, applying an animation will overwrite this base value and apply the animated value for as long as your animation continues to run.

For an animated property, the base value can still have an effect on the animation's behavior, if that animation does not explicitly specify both **From** and **To**, or if the

animation reverts the property to its base value when completed. In these cases, once an animation is no longer running, the rest of the precedence is used again.

However, an animation that specifies a **To** with a **HoldEnd** behavior can override a local value until the animation is removed, even when it visually appears to be stopped. Conceptually this is like an animation that's running forever even if there is not a visual animation in the UI.

Multiple animations can be applied to a single property. Each of these animations might have been defined to replace base values that came from different points in the value precedence. However, these animations will all be running simultaneously at run time, and that often means that they must combine their values because each animation has equal influence on the value. This depends on exactly how the animations are defined, and the type of the value that is being animated.

For more info, see Storyboarded animations.

Default values

Establishing the default value for a dependency property with a **PropertyMetadata** value is explained in more detail in the Custom dependency properties topic.

Dependency properties still have default values even if those default values weren't explicitly defined in that property's metadata. Unless they have been changed by metadata, default values for the Windows Runtime dependency properties are generally one of the following:

- A property that uses a run-time object or the basic **Object** type (a *reference type*) has a default value of **null**. For example, **DataContext** is **null** until it's deliberately set or is inherited.
- A property that uses a basic value such as numbers or a Boolean value (a *value type*) uses an expected default for that value. For example, 0 for integers and floating-point numbers, **false** for a Boolean.
- A property that uses a Windows Runtime structure has a default value that's obtained by calling that structure's implicit default constructor. This constructor uses the defaults for each of the basic value fields of the structure. For example, a default for a **Point** value is initialized with its **X** and **Y** values as 0.
- A property that uses an enumeration has a default value of the first defined member in that enumeration. Check the reference for specific enumerations to see what the default value is.
- A property that uses a string (**System.String** for .NET, **Platform::String** for C++/CX) has a default value of an empty string ("").

- Collection properties aren't typically implemented as dependency properties, for reasons discussed further on in this topic. But if you implement a custom collection property and you want it to be a dependency property, make sure to avoid an *unintentional singleton* as described near the end of Custom dependency properties.

Property functionality provided by a dependency property

Data binding

A dependency property can have its value set through applying a data binding. Data binding uses the {Binding} markup extension syntax in XAML, {x:Bind} markup extension or the **Binding** class in code. For a databound property, the final property value determination is deferred until run time. At that time the value is obtained from a data source. The role that the dependency property system plays here is enabling a placeholder behavior for operations like loading XAML when the value is not yet known, and then supplying the value at run time by interacting with the Windows Runtime data binding engine.

The following example sets the **Text** value for a **TextBlock** element, using a binding in XAML. The binding uses an inherited data context and an object data source. (Neither of these is shown in the shortened example; for a more complete sample that shows context and source, see Data binding in depth.)

XAML

```
<Canvas>
  <TextBlock Text="{Binding Team.TeamName}"/>
</Canvas>
```

You can also establish bindings using code rather than XAML. See **SetBinding**.

> **Note**
>
> Bindings like this are treated as a local value for purposes of dependency property value precedence. If you set another local value for a property that originally held a **Binding** value, you will overwrite the binding entirely, not just the binding's run-time value. {x:Bind} Bindings are implemented using generated code that will set a local value for the property. If you set a local value for a property that is using

{x:Bind}, then that value will be replaced the next time the binding is evaluated, such as when it observes a property change on its source object.

Binding sources, binding targets, the role of FrameworkElement

To be the source of a binding, a property does not need to be a dependency property; you can generally use any property as a binding source, although this depends on your programming language and each has certain edge cases. However, to be the target of a {Binding} markup extension or **Binding**, that property must be a dependency property. {x:Bind} does not have this requirement as it uses generated code to apply its binding values.

If you are creating a binding in code, note that the **SetBinding** API is defined only for **FrameworkElement**. However, you can create a binding definition using **BindingOperations** instead, and thus reference any **DependencyObject** property.

For either code or XAML, remember that **DataContext** is a **FrameworkElement** property. By using a form of parent-child property inheritance (typically established in XAML markup), the binding system can resolve a **DataContext** that exists on a parent element. This inheritance can evaluate even if the child object (which has the target property) is not a **FrameworkElement** and therefore does not hold its own **DataContext** value. However, the parent element being inherited must be a **FrameworkElement** in order to set and hold the **DataContext**. Alternatively, you must define the binding such that it can function with a **null** value for **DataContext**.

Wiring the binding is not the only thing that's needed for most data binding scenarios. For a one-way or two-way binding to be effective, the source property must support change notifications that propagate to the binding system and thus the target. For custom binding sources, this means that the property must be a dependency property, or the object must support **INotifyPropertyChanged**. Collections should support **INotifyCollectionChanged**. Certain classes support these interfaces in their implementations so that they are useful as base classes for data binding scenarios; an example of such a class is **ObservableCollection<T>**. For more information on data binding and how data binding relates to the property system, see Data binding in depth.

> **Note**
>
> The types listed here support Microsoft .NET data sources. C++/CX data sources use different interfaces for change notification or observable behavior, see **Data binding in depth**.

Styles and templates

Styles and templates are two of the scenarios for properties being defined as dependency properties. Styles are useful for setting properties that define the app's UI. Styles are defined as resources in XAML, either as an entry in a **Resources** collection, or in separate XAML files such as theme resource dictionaries. Styles interact with the property system because they contain setters for properties. The most important property here is the **Control.Template** property of a **Control**: it defines most of the visual appearance and visual state for a **Control**. For more info on styles, and some example XAML that defines a **Style** and uses setters, see Styling controls.

Values that come from styles or templates are deferred values, similar to bindings. This is so that control users can re-template controls or redefine styles. And that's why property setters in styles can only act on dependency properties, not ordinary properties.

Storyboarded animations

You can animate a dependency property's value using a storyboarded animation. Storyboarded animations in the Windows Runtime are not merely visual decorations. It's more useful to think of animations as being a state machine technique that can set the values of individual properties or of all properties and visuals of a control, and change these values over time.

To be animated, the animation's target property must be a dependency property. Also, to be animated, the target property's value type must be supported by one of the existing **Timeline**-derived animation types. Values of **Color**, **Double** and **Point** can be animated using either interpolation or keyframe techniques. Most other values can be animated using discrete **Object** key frames.

When an animation is applied and running, the animated value operates at a higher precedence than any value (such as a local value) that the property otherwise has. Animations also have an optional **HoldEnd** behavior that can cause animations to apply to property values even if the animation visually appears to be stopped.

The state machine principle is embodied by the use of storyboarded animations as part of the **VisualStateManager** state model for controls. For more info on storyboarded animations, see Storyboarded animations. For more info on **VisualStateManager** and defining visual states for controls, see Storyboarded animations for visual states or Control templates.

Property-changed behavior

Property-changed behavior is the origin of the "dependency" part of dependency property terminology. Maintaining valid values for a property when another property can influence the first property's value is a difficult development problem in many frameworks. In the Windows Runtime property system, each dependency property can specify a callback that is invoked whenever its property value changes. This callback can be used to notify or change related property values, in a generally synchronous manner. Many existing dependency properties have a property-changed behavior. You can also add similar callback behavior to custom dependency properties, and implement your own property-changed callbacks. See Custom dependency properties for an example.

Windows 10 introduces the **RegisterPropertyChangedCallback** method. This enables application code to register for change notifications when the specified dependency property is changed on an instance of **DependencyObject**.

Default value and ClearValue

A dependency property can have a default value defined as part of its property metadata. For a dependency property, its default value doesn't become irrelevant after the property's been set the first time. The default value might apply again at run-time whenever some other determinant in value precedence disappears. (Dependency property value precedence is discussed in the next section.) For example, you might deliberately remove a style value or an animation that applies to a property, but you want the value to be a reasonable default after you do so. The dependency property default value can provide this value, without needing to specifically set each property's value as an extra step.

You can deliberately set a property to the default value even after you have already set it with a local value. To reset a value to be the default again, and also to enable other participants in precedence that might override the default but not a local value, call the **ClearValue** method (reference the property to clear as a method parameter). You don't always want the property to literally use the default value, but clearing the local value and reverting to the default value might enable another item in precedence that you want to act now, such as using the value that came from a style setter in a control template.

DependencyObject and threading

All **DependencyObject** instances must be created on the UI thread which is associated with the current **Window** that is shown by a Windows Runtime app. Although each

DependencyObject must be created on the main UI thread, the objects can be accessed using a dispatcher reference from other threads, by accessing the **Dispatcher** property. Then you can call methods such as **RunAsync** on the **CoreDispatcher** object, and execute your code within the rules of thread restrictions on the UI thread.

The threading aspects of **DependencyObject** are relevant because it generally means that only code that runs on the UI thread can change or even read the value of a dependency property. Threading issues can usually be avoided in typical UI code that makes correct use of **async** patterns and background worker threads. You typically only run into **DependencyObject**-related threading issues if you are defining your own **DependencyObject** types and you attempt to use them for data sources or other scenarios where a **DependencyObject** isn't necessarily appropriate.

Related topics

Conceptual material

- Custom dependency properties
- Attached properties overview
- Data binding in depth
- Storyboarded animations
- Creating Windows Runtime components
- XAML user and custom controls sample

APIs related to dependency properties

- **DependencyObject**
- **DependencyProperty**

Custom dependency properties

Article • 10/20/2022

Here we explain how to define and implement your own dependency properties for a Windows Runtime app using C++, C#, or Visual Basic. We list reasons why app developers and component authors might want to create custom dependency properties. We describe the implementation steps for a custom dependency property, as well as some best practices that can improve performance, usability, or versatility of the dependency property.

Prerequisites

We assume that you have read the Dependency properties overview and that you understand dependency properties from the perspective of a consumer of existing dependency properties. To follow the examples in this topic, you should also understand XAML and know how to write a basic Windows Runtime app using C++, C#, or Visual Basic.

What is a dependency property?

To support styling, data binding, animations, and default values for a property, then it should be implemented as a dependency property. Dependency property values are not stored as fields on the class, they are stored by the xaml framework, and are referenced using a key, which is retrieved when the property is registered with the Windows Runtime property system by calling the **DependencyProperty.Register** method. Dependency properties can be used only by types deriving from **DependencyObject**. But **DependencyObject** is quite high in the class hierarchy, so the majority of classes that are intended for UI and presentation support can support dependency properties. For more information about dependency properties and some of the terminology and conventions used for describing them in this documentation, see Dependency properties overview.

Examples of dependency properties in the Windows Runtime are: **Control.Background**, **FrameworkElement.Width**, and **TextBox.Text**, among many others.

Convention is that each dependency property exposed by a class has a corresponding **public static readonly** property of type **DependencyProperty** that is exposed on that same class the provides the identifier for the dependency property. The identifier's name follows this convention: the name of the dependency property, with the string "Property" added to the end of the name. For example, the corresponding

DependencyProperty identifier for the **Control.Background** property is Control.BackgroundProperty. The identifier stores the information about the dependency property as it was registered, and can then be used for other operations involving the dependency property, such as calling SetValue.

Property wrappers

Dependency properties typically have a wrapper implementation. Without the wrapper, the only way to get or set the properties would be to use the dependency property utility methods GetValue and SetValue and to pass the identifier to them as a parameter. This is a rather unnatural usage for something that is ostensibly a property. But with the wrapper, your code and any other code that references the dependency property can use a straightforward object-property syntax that is natural for the language you're using.

If you implement a custom dependency property yourself and want it to be public and easy to call, define the property wrappers too. The property wrappers are also useful for reporting basic information about the dependency property to reflection or static analysis processes. Specifically, the wrapper is where you place attributes such as ContentPropertyAttribute.

When to implement a property as a dependency property

Whenever you implement a public read/write property on a class, as long as your class derives from DependencyObject, you have the option to make your property work as a dependency property. Sometimes the typical technique of backing your property with a private field is adequate. Defining your custom property as a dependency property is not always necessary or appropriate. The choice will depend on the scenarios that you intend your property to support.

You might consider implementing your property as a dependency property when you want it to support one or more of these features of the Windows Runtime or of Windows Runtime apps:

- Setting the property through a Style
- Acting as valid target property for data binding with {Binding}
- Supporting animated values through a Storyboard
- Reporting when the value of the property has been changed by:
  - Actions taken by the property system itself
  - The environment

    - User actions
    - Reading and writing styles

Checklist for defining a dependency property

Defining a dependency property can be thought of as a set of concepts. These concepts are not necessarily procedural steps, because several concepts can be addressed in a single line of code in the implementation. This list gives just a quick overview. We'll explain each concept in more detail later in this topic, and we'll show you example code in several languages.

- Register the property name with the property system (call **Register**), specifying an owner type and the type of the property value.
    - There's a required parameter for **Register** that expects property metadata. Specify **null** for this, or if you want property-changed behavior, or a metadata-based default value that can be restored by calling **ClearValue**, specify an instance of **PropertyMetadata**.
- Define a **DependencyProperty** identifier as a **public static readonly** property member on the owner type.
- Define a wrapper property, following the property accessor model that's used in the language you are implementing. The wrapper property name should match the *name* string that you used in **Register**. Implement the **get** and **set** accessors to connect the wrapper with the dependency property that it wraps, by calling **GetValue** and **SetValue** and passing your own property's identifier as a parameter.
- (Optional) Place attributes such as **ContentPropertyAttribute** on the wrapper.

> **Note**
>
> If you are defining a custom attached property, you generally omit the wrapper. Instead, you write a different style of accessor that a XAML processor can use. See Custom attached properties.

Registering the property

For your property to be a dependency property, you must register the property into a property store maintained by the Windows Runtime property system. To register the property, you call the **Register** method.

For Microsoft .NET languages (C# and Microsoft Visual Basic) you call **Register** within the body of your class (inside the class, but outside any member definitions). The

identifier is provided by the Register method call, as the return value. The Register call is typically made as a static constructor or as part of the initialization of a **public static readonly** property of type DependencyProperty as part of your class. This property exposes the identifier for your dependency property. Here are examples of the Register call.

> ⓘ Note
>
> Registering the dependency property as part of the identifier property definition is the typical implementation, but you can also register a dependency property in the class static constructor. This approach may make sense if you need more than one line of code to initialize the dependency property.

For C++/CX, you have options for how you split the implementation between the header and the code file. The typical split is to declare the identifier itself as **public static** property in the header, with a **get** implementation but no **set**. The **get** implementation refers to a private field, which is an uninitialized DependencyProperty instance. You can also declare the wrappers and the **get** and **set** implementations of the wrapper. In this case the header includes some minimal implementation. If the wrapper needs Windows Runtime attribution, attribute in the header too. Put the Register call in the code file, within a helper function that only gets run when the app initializes the first time. Use the return value of **Register** to fill the static but uninitialized identifiers that you declared in the header, which you initially set to **nullptr** at the root scope of the implementation file.

C#

```
public static readonly DependencyProperty LabelProperty =
DependencyProperty.Register(
  nameof(Label),
  typeof(String),
  typeof(ImageWithLabelControl),
  new PropertyMetadata(null)
);
```

> ⓘ Note
>
> For the C++/CX code, the reason why you have a private field and a public read-only property that surfaces the DependencyProperty is so that other callers who use your dependency property can also use property-system utility APIs that require the identifier to be public. If you keep the identifier private, people can't use these utility APIs. Examples of such API and scenarios include GetValue or SetValue by choice, ClearValue, GetAnimationBaseValue, SetBinding, and

Setter.Property. You can't use a public field for this, because Windows Runtime metadata rules don't allow for public fields.

Dependency property name conventions

There are naming conventions for dependency properties; follow them in all but exceptional circumstances. The dependency property itself has a basic name ("Label" in the preceding example) that is given as the first parameter of Register. The name must be unique within each registering type, and the uniqueness requirement also applies to any inherited members. Dependency properties inherited through base types are considered to be part of the registering type already; names of inherited properties cannot be registered again.

> ⚠ **Warning**
>
> Although the name you provide here can be any string identifier that is valid in programming for your language of choice, you usually want to be able to set your dependency property in XAML too. To be set in XAML, the property name you choose must be a valid XAML name. For more info, see XAML overview.

When you create the identifier property, combine the name of the property as you registered it with the suffix "Property" ("LabelProperty", for example). This property is your identifier for the dependency property, and it is used as an input for the SetValue and GetValue calls you make in your own property wrappers. It is also used by the property system and other XAML processors such as {x:Bind}

Implementing the wrapper

Your property wrapper should call GetValue in the **get** implementation, and SetValue in the **set** implementation.

> ⚠ **Warning**
>
> In all but exceptional circumstances, your wrapper implementations should perform only the GetValue and SetValue operations. Otherwise, you'll get different behavior when your property is set via XAML versus when it is set via code. For efficiency, the XAML parser bypasses wrappers when setting dependency properties; and talks to the backing store via **SetValue**.

C#

```
public String Label
{
    get { return (String)GetValue(LabelProperty); }
    set { SetValue(LabelProperty, value); }
}
```

Property metadata for a custom dependency property

When property metadata is assigned to a dependency property, the same metadata is applied to that property for every instance of the property-owner type or its subclasses. In property metadata, you can specify two behaviors:

- A default value that the property system assigns to all cases of the property.
- A static callback method that is automatically invoked within the property system whenever a property value change is detected.

Calling Register with property metadata

In the previous examples of calling DependencyProperty.Register, we passed a null value for the *propertyMetadata* parameter. To enable a dependency property to provide a default value or use a property-changed callback, you must define a PropertyMetadata instance that provides one or both of these capabilities.

Typically you provide a PropertyMetadata as an inline-created instance, within the parameters for DependencyProperty.Register.

> ⓘ Note
>
> If you are defining a CreateDefaultValueCallback implementation, you must use the utility method PropertyMetadata.Create rather than calling a PropertyMetadata constructor to define the **PropertyMetadata** instance.

This next example modifies the previously shown DependencyProperty.Register examples by referencing a PropertyMetadata instance with a PropertyChangedCallback value. The implementation of the "OnLabelChanged" callback will be shown later in this section.

C#

```
public static readonly DependencyProperty LabelProperty =
DependencyProperty.Register(
  nameof(Label),
  typeof(String),
  typeof(ImageWithLabelControl),
  new PropertyMetadata(null,new PropertyChangedCallback(OnLabelChanged))
);
```

Default value

You can specify a default value for a dependency property such that the property always returns a particular default value when it is unset. This value can be different than the inherent default value for the type of that property.

If a default value is not specified, the default value for a dependency property is null for a reference type, or the default of the type for a value type or language primitive (for example, 0 for an integer or an empty string for a string). The main reason for establishing a default value is that this value is restored when you call **ClearValue** on the property. Establishing a default value on a per-property basis might be more convenient than establishing default values in constructors, particularly for value types. However, for reference types, make sure that establishing a default value does not create an unintentional singleton pattern. For more info, see Best practices later in this topic

C++/WinRT

```
// ImageWithLabelControl.cpp
...
Windows::UI::Xaml::DependencyProperty ImageWithLabelControl::m_labelProperty
=
    Windows::UI::Xaml::DependencyProperty::Register(
        L"Label",
        winrt::xaml_typename<winrt::hstring>(),

winrt::xaml_typename<ImageWithLabelControlApp::ImageWithLabelControl>(),
        Windows::UI::Xaml::PropertyMetadata{ winrt::box_value(L"default
label"), Windows::UI::Xaml::PropertyChangedCallback{
&ImageWithLabelControl::OnLabelChanged } }
);
...
```

> **Note**

Do not register with a default value of UnsetValue. If you do, it will confuse property consumers and will have unintended consequences within the property system.

CreateDefaultValueCallback

In some scenarios, you are defining dependency properties for objects that are used on more than one UI thread. This might be the case if you are defining a data object that is used by multiple apps, or a control that you use in more than one app. You can enable the exchange of the object between different UI threads by providing a CreateDefaultValueCallback implementation rather than a default value instance, which is tied to the thread that registered the property. Basically a CreateDefaultValueCallback defines a factory for default values. The value returned by **CreateDefaultValueCallback** is always associated with the current UI **CreateDefaultValueCallback** thread that is using the object.

To define metadata that specifies a CreateDefaultValueCallback, you must call PropertyMetadata.Create to return a metadata instance; the PropertyMetadata constructors do not have a signature that includes a **CreateDefaultValueCallback** parameter.

The typical implementation pattern for a CreateDefaultValueCallback is to create a new DependencyObject class, set the specific property value of each property of the **DependencyObject** to the intended default, and then return the new class as an **Object** reference via the return value of the **CreateDefaultValueCallback** method.

Property-changed callback method

You can define a property-changed callback method to define your property's interactions with other dependency properties, or to update an internal property or state of your object whenever the property changes. If your callback is invoked, the property system has determined that there is an effective property value change. Because the callback method is static, the *d* parameter of the callback is important because it tells you which instance of the class has reported a change. A typical implementation uses the NewValue property of the event data and processes that value in some manner, usually by performing some other change on the object passed as *d*. Additional responses to a property change are to reject the value reported by **NewValue**, to restore OldValue, or to set the value to a programmatic constraint applied to the **NewValue**.

This next example shows a PropertyChangedCallback implementation. It implements the method you saw referenced in the previous Register examples, as part of the construction arguments for the PropertyMetadata. The scenario addressed by this callback is that the class also has a calculated read-only property named "HasLabelValue" (implementation not shown). Whenever the "Label" property gets reevaluated, this callback method is invoked, and the callback enables the dependent calculated value to remain in synchronization with changes to the dependency property.

C#

```
private static void OnLabelChanged(DependencyObject d,
DependencyPropertyChangedEventArgs e) {
    ImageWithLabelControl iwlc = d as ImageWithLabelControl; //null checks
omitted
    String s = e.NewValue as String; //null checks omitted
    if (s == String.Empty)
    {
        iwlc.HasLabelValue = false;
    } else {
        iwlc.HasLabelValue = true;
    }
}
```

Property changed behavior for structures and enumerations

If the type of a DependencyProperty is an enumeration or a structure, the callback may be invoked even if the internal values of the structure or the enumeration value did not change. This is different from a system primitive such as a string where it only is invoked if the value changed. This is a side effect of box and unbox operations on these values that is done internally. If you have a PropertyChangedCallback method for a property where your value is an enumeration or structure, you need to compare the OldValue and NewValue by casting the values yourself and using the overloaded comparison operators that are available to the now-cast values. Or, if no such operator is available (which might be the case for a custom structure), you may need to compare the individual values. You would typically choose to do nothing if the result is that the values have not changed.

C#

```
private static void OnVisibilityValueChanged(DependencyObject d,
DependencyPropertyChangedEventArgs e) {
    if ((Visibility)e.NewValue != (Visibility)e.OldValue)
    {
        //value really changed, invoke your changed logic here
```

```
    } // else this was invoked because of boxing, do nothing
}
```

Best practices

Keep the following considerations in mind as best practices when as you define your custom dependency property.

DependencyObject and threading

All **DependencyObject** instances must be created on the UI thread which is associated with the current **Window** that is shown by a Windows Runtime app. Although each **DependencyObject** must be created on the main UI thread, the objects can be accessed using a dispatcher reference from other threads, by calling **Dispatcher**.

The threading aspects of **DependencyObject** are relevant because it generally means that only code that runs on the UI thread can change or even read the value of a dependency property. Threading issues can usually be avoided in typical UI code that makes correct use of **async** patterns and background worker threads. You typically only run into **DependencyObject**-related threading issues if you are defining your own **DependencyObject** types and you attempt to use them for data sources or other scenarios where a **DependencyObject** isn't necessarily appropriate.

Avoiding unintentional singletons

An unintentional singleton can happen if you are declaring a dependency property that takes a reference type, and you call a constructor for that reference type as part of the code that establishes your **PropertyMetadata**. What happens is that all usages of the dependency property share just one instance of **PropertyMetadata** and thus try to share the single reference type you constructed. Any subproperties of that value type that you set through your dependency property then propagate to other objects in ways you may not have intended.

You can use class constructors to set initial values for a reference-type dependency property if you want a non-null value, but be aware that this would be considered a local value for purposes of Dependency properties overview. It might be more appropriate to use a template for this purpose, if your class supports templates. Another way to avoid a singleton pattern, but still provide a useful default, is to expose a static property on the reference type that provides a suitable default for the values of that class.

Collection-type dependency properties

Collection-type dependency properties have some additional implementation issues to consider.

Collection-type dependency properties are relatively rare in the Windows Runtime API. In most cases, you can use collections where the items are a **DependencyObject** subclass, but the collection property itself is implemented as a conventional CLR or C++ property. This is because collections do not necessarily suit some typical scenarios where dependency properties are involved. For example:

- You do not typically animate a collection.
- You do not typically prepopulate the items in a collection with styles or a template.
- Although binding to collections is a major scenario, a collection does not need to be a dependency property to be a binding source. For binding targets, it is more typical to use subclasses of **ItemsControl** or **DataTemplate** to support collection items, or to use view-model patterns. For more info about binding to and from collections, see Data binding in depth.
- Notifications for collection changes are better addressed through interfaces such as **INotifyPropertyChanged** or **INotifyCollectionChanged**, or by deriving the collection type from **ObservableCollection<T>**.

Nevertheless, scenarios for collection-type dependency properties do exist. The next three sections provide some guidance on how to implement a collection-type dependency property.

Initializing the collection

When you create a dependency property, you can establish a default value by means of dependency property metadata. But be careful to not use a singleton static collection as the default value. Instead, you must deliberately set the collection value to a unique (instance) collection as part of class-constructor logic for the owner class of the collection property.

C#

```
// WARNING - DO NOT DO THIS
public static readonly DependencyProperty ItemsProperty =
DependencyProperty.Register(
  nameof(Items),
  typeof(IList<object>),
  typeof(ImageWithLabelControl),
  new PropertyMetadata(new List<object>())
);
```

```
// DO THIS Instead
public static readonly DependencyProperty ItemsProperty =
DependencyProperty.Register(
  nameof(Items),
  typeof(IList<object>),
  typeof(ImageWithLabelControl),
  new PropertyMetadata(null)
);

public ImageWithLabelControl()
{
    // Need to initialize in constructor instead
    Items = new List<object>();
}
```

A DependencyProperty and its PropertyMetadata's default value are part of the static definition of the DependencyProperty. By providing a default collection (or other instanced) value as the default value, it will be shared across all instances of your class instead of each class having its own collection, as would typically be desired.

Change notifications

Defining the collection as a dependency property does not automatically provide change notification for the items in the collection by virtue of the property system invoking the "PropertyChanged" callback method. If you want notifications for collections or collection items—for example, for a data-binding scenario— implement the **INotifyPropertyChanged** or **INotifyCollectionChanged** interface. For more info, see Data binding in depth.

Dependency property security considerations

Declare dependency properties as public properties. Declare dependency property identifiers as **public static readonly** members. Even if you attempt to declare other access levels permitted by a language (such as **protected**), a dependency property can always be accessed through the identifier in combination with the property-system APIs. Declaring the dependency property identifier as internal or private will not work, because then the property system cannot operate properly.

Wrapper properties are really just for convenience, Security mechanisms applied to the wrappers can be bypassed by calling **GetValue** or **SetValue** instead. So keep wrapper properties public; otherwise you just make your property harder for legitimate callers to use without providing any real security benefit.

The Windows Runtime does not provide a way to register a custom dependency property as read-only.

Dependency properties and class constructors

There is a general principle that class constructors should not call virtual methods. This is because constructors can be called to accomplish base initialization of a derived class constructor, and entering the virtual method through the constructor might occur when the object instance being constructed is not yet completely initialized. When you derive from any class that already derives from **DependencyObject**, remember that the property system itself calls and exposes virtual methods internally as part of its services. To avoid potential problems with run-time initialization, don't set dependency property values within constructors of classes.

Registering the dependency properties for C++/CX apps

The implementation for registering a property in C++/CX is trickier than C#, both because of the separation into header and implementation file and also because initialization at the root scope of the implementation file is a bad practice. (Visual C++ component extensions (C++/CX) puts static initializer code from the root scope directly into **DllMain**, whereas C# compilers assign the static initializers to classes and thus avoid **DllMain** load lock issues.). The best practice here is to declare a helper function that does all your dependency property registration for a class, one function per class. Then for each custom class your app consumes, you'll have to reference the helper registration function that's exposed by each custom class you want to use. Call each helper registration function once as part of the **Application constructor** (`App::App()`), prior to `InitializeComponent`. That constructor only runs when the app is really referenced for the first time, it won't run again if a suspended app resumes, for example. Also, as seen in the previous C++ registration example, the **nullptr** check around each **Register** call is important: it's insurance that no caller of the function can register the property twice. A second registration call would probably crash your app without such a check because the property name would be a duplicate. You can see this implementation pattern in the XAML user and custom controls sample if you look at the code for the C++/CX version of the sample.

Related topics

- **DependencyObject**
- **DependencyProperty.Register**
- Dependency properties overview

- XAML user and custom controls sample

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Attached properties overview

Article • 10/20/2022

An *attached property* is a XAML concept. Attached properties enable additional property/value pairs to be set on an object, but the properties are not part of the original object definition. Attached properties are typically defined as a specialized form of dependency property that doesn't have a conventional property wrapper in the owner type's object model.

Prerequisites

We assume that you understand the basic concept of dependency properties, and have read Dependency properties overview.

Attached properties in XAML

In XAML, you set attached properties by using the syntax *AttachedPropertyProvider.PropertyName*. Here is an example of how you can set **Canvas.Left** in XAML.

XAML

```
<Canvas>
  <Button Canvas.Left="50">Hello</Button>
</Canvas>
```

> **Note**
>
> We're just using **Canvas.Left** as an example attached property without fully explaining why you'd use it. If you want to know more about what **Canvas.Left** is for and how **Canvas** handles its layout children, see the **Canvas** reference topic or **Define layouts with XAML**.

Why use attached properties?

Attached properties are a way to escape the coding conventions that might prevent different objects in a relationship from communicating information to each other at run time. It's certainly possible to put properties on a common base class so that each

object could just get and set that property. But eventually the sheer number of scenarios where you might want to do this will bloat your base classes with shareable properties. It might even introduce cases where there might just be two of hundreds of descendants trying to use a property. That's not good class design. To address this, the attached property concept enables an object to assign a value for a property that its own class structure doesn't define. The defining class can read the value from child objects at run time after the various objects are created in an object tree.

For example, child elements can use attached properties to inform their parent element of how they are to be presented in the UI. This is the case with the Canvas.Left attached property. **Canvas.Left** is created as an attached property because it is set on elements that are contained within a Canvas element, rather than on the **Canvas** itself. Any possible child element then uses **Canvas.Left** and Canvas.Top to specify its layout offset within the **Canvas** layout container parent. Attached properties make it possible for this to work without cluttering the base element's object model with lots of properties that each apply to only one of the many possible layout containers. Instead, many of the layout containers implement their own attached property set.

To implement the attached property, the Canvas class defines a static DependencyProperty field named Canvas.LeftProperty. Then, **Canvas** provides the SetLeft and GetLeft methods as public accessors for the attached property, to enable both XAML setting and run-time value access. For XAML and for the dependency property system, this set of APIs satisfies a pattern that enables a specific XAML syntax for attached properties, and stores the value in the dependency property store.

How the owning type uses attached properties

Although attached properties can be set on any XAML element (or any underlying DependencyObject), that doesn't automatically mean that setting the property produces a tangible result, or that the value is ever accessed. The type that defines the attached property typically follows one of these scenarios:

- The type that defines the attached property is the parent in a relationship of other objects. The child objects will set values for the attached property. The attached property owner type has some innate behavior that iterates through its child elements, obtains the values, and acts on those values at some point in object lifetime (a layout action, SizeChanged, etc.)
- The type that defines the attached property is used as the child element for a variety of possible parent elements and content models, but the info isn't necessarily layout info.
- The attached property reports info to a service, not to another UI element.